

COMANDOS_DEPLOYMENT

Guia operacional (copiar e colar) com **todos os comandos do deployment**, organizados por passo, com indicação de terminal, objetivo, verificação e output esperado.

Legenda rápida de terminal

-  **LOCAL** = seu computador de desenvolvimento (onde está seu clone Git)
 -  **SERVIDOR (SSH)** = terminal conectado no servidor de produção (`ssh deploy@SEU_SERVIDOR`)
-
-

PASSO 1: Preparar arquivos para commit

 **EXECUTAR NO SEU COMPUTADOR LOCAL**

O que este bloco faz

Copia arquivos do projeto para o repositório Git, revisa alterações e publica branch no GitHub.

```
# 1) Definir caminhos
SOURCE_DIR="/home/ubuntu/cidadesverdes"
TARGET_REPO="/caminho/do/seu/repo-git" # ajuste este caminho

# 2) Entrar no repo existente
cd "$TARGET_REPO"

# 3) (Opcional, recomendado) Criar branch de release/deploy
git checkout -b chore/deploy-api-ambiente-8101

# 4) Copiar arquivos mantendo estrutura
rsync -av --delete \
  --exclude='.git' \
  --exclude='.venv' \
  --exclude='__pycache__' \
  --exclude='*.pyc' \
  "$SOURCE_DIR/" "$TARGET_REPO/"

# 5) Verificar o que mudou
git status
git diff --stat

# 6) (Opcional) Revisar conteúdo exato alterado
git diff

# 7) Preparar commit
git add .
git status

# 8) Commit
git commit -m "feat: adiciona API ambiente (8101) com Firebird e automação de deploy"

# 9) Push da branch
git push -u origin chore/deploy-api-ambiente-8101
```

Output esperado

- `rsync` listando arquivos copiados
- `git status` com `Changes to be committed`
- `git push` concluindo sem erro

Pausa para verificação

```
git log --oneline -n 3
git branch -vv
```

PASSO 2: Atualizar servidor via Git Pull

EXECUTAR NO TERMINAL DO SERVIDOR (SSH)

O que este bloco faz

Conecta no servidor e sincroniza código de produção com o GitHub.

```
# 1) Acessar servidor
ssh deploy@SEU_SERVIDOR

# 2) Ir para projeto
cd /home/deploy/var/www/sistema-gestao/cidadesverdes

# 3) Verificar branch atual
git branch --show-current

# 4) Atualizar referências remotas
git fetch --all --prune

# 5) Deploy direto na branch principal (ex.: main)
git pull origin main

# 6) OU deploy por branch específica
git checkout chore/deploy-api-ambiente-8101
git pull origin chore/deploy-api-ambiente-8101
```

Output esperado

- `git pull` retornando `Already up to date.` ou lista de arquivos atualizados.

Pausa para verificação

```
git log --oneline -n 5
ls -la app_ambiente scripts systemd sql
```

PASSO 3: Instalar Firebird 3.0.13

EXECUTAR NO TERMINAL DO SERVIDOR (SSH)

O que este bloco faz

Instala/valida Firebird, inicia serviço e confirma versão e porta.

```
cd /home/deploy/var/www/sistema-gestao/cidadesverdes
chmod +x scripts/setup_firebird.sh
./scripts/setup_firebird.sh
```

Pausa para verificação

```
# Versão instalada
dpkg-query -W -f='${Version}\n' firebird3.0-server

# Serviço ativo
sudo systemctl status firebird3.0 --no-pager

# Porta 3050 escutando
sudo ss -ltnp | grep 3050 || true
```

Configuração inicial Firebird

```
# Ver senha atual do SYSDBA
sudo cat /etc/firebird/3.0/SYSDBA.password

# (Opcional) alterar senha do SYSDBA
sudo -u firebird gsec -user sysdba -password masterkey -modify sysdba -pw 'NOVA_SENHA_FORTE_AQUI'
```

Output esperado

- firebird3.0 em active (running)
- senha visível em /etc/firebird/3.0/SYSDBA.password

PASSO 4: Criar diretório do banco

EXECUTAR NO TERMINAL DO SERVIDOR (SSH)

O que este bloco faz

Cria pasta física do banco Firebird e aplica permissões corretas.

```
sudo mkdir -p /home/deploy/var/databases/
sudo chown -R firebird:firebird /home/deploy/var/databases/
sudo chmod -R 775 /home/deploy/var/databases/
```

Pausa para verificação

```
ls -ld /home/deploy/var/databases/
namei -l /home/deploy/var/databases/
```

Output esperado

- Permissão final parecida com drwxrwxr-x e owner firebird:firebird .

PASSO 5: Configurar variáveis de ambiente (.env)

EXECUTAR NO TERMINAL DO SERVIDOR (SSH)

O que este bloco faz

Cria .env , define credenciais/configurações e endurece permissões do arquivo.

```
cd /home/deploy/var/www/sistema-gestao/cidadesverdes
cp .env.example .env
nano .env
```

Conteúdo de referência para o `.env`

```
# Firebird
FIREBIRD_HOST=127.0.0.1
FIREBIRD_PORT=3050
FIREBIRD_DATABASE=/home/deploy/var/databases/cidadesverdes.fdb
FIREBIRD_USER=SYSDBA
FIREBIRD_PASSWORD=COLOQUE_SENHA_FORTE_AQUI
FIREBIRD_CHARSET=UTF8

# API Ambiente
API_HOST=0.0.0.0
API_PORT=8101
CORS_ALLOW_ORIGINS=*
```

Gerar senha segura

```
# Opção 1 (OpenSSL)
openssl rand -base64 24

# Opção 2 (Python)
python3 - << 'PY'
import secrets
print(secrets.token_urlsafe(24))
PY
```

Segurança do `.env`

```
chmod 600 .env
ls -l .env
```

Pausa para verificação

```
# Conferir nomes das variáveis sem vaziar segredo
grep -E '^(FIREBIRD_|API_|CORS_)' .env | sed 's/=.*/=*** oculto **/'
```

Output esperado

- `.env` presente com permissões `-rw-----`.

PASSO 6: Criar banco e schema

EXECUTAR NO TERMINAL DO SERVIDOR (SSH)

O que este bloco faz

Ativa venv, instala dependências e cria banco + tabela + sequence + trigger.

```
cd /home/deploy/var/www/sistema-gestao/cidadesverdes
source .venv/bin/activate
pip install -r requirements.txt
python3 scripts/create_database.py
```

Output esperado

- Banco criado em: ... (primeira execução)
- Tabela ambiente criada.
- Sequence ambiente_seq criada.
- Trigger bi_ambiente criada.
- Inicialização concluída com sucesso.

Pausa para verificação

```
ls -lh /home/deploy/var/databases/cidadesverdes.fdb

isql-fb -user SYSDBA -password 'SENHA_AQUI' /home/deploy/var/databases/cidades-
verdes.fdb << 'SQL'
SHOW TABLES;
SHOW SEQUENCES;
SHOW TRIGGERS;
QUIT;
SQL
```

PASSO 7: Testar API manualmente

EXECUTAR NO TERMINAL DO SERVIDOR (SSH)

O que este bloco faz

Sobe API em modo manual para validar endpoints antes do systemd.

```
cd /home/deploy/var/www/sistema-gestao/cidadesverdes
source .venv/bin/activate
./scripts/run_api_ambiente.sh
```

EM OUTRO TERMINAL SSH (enquanto a API está rodando)

```
# Healthcheck
curl -sS http://127.0.0.1:8101/ | jq .

# Inserir
curl -sS -X POST 'http://127.0.0.1:8101/ambiente' \
-H 'Content-Type: application/json' \
-d '{
  "temperatura": 25.4,
  "umidade": 63.1,
  "co2": 412.7,
  "bairro": "Centro",
  "cidade": "Sao Paulo",
  "estado": "SP",
  "pais": "Brasil",
  "latitude": "-23.5505",
  "longitude": "-46.6333",
  "data": "2026-05-07",
  "hora": "10:30:00",
  "origemdaleitura": "sensor_api"
}' | jq .

# Listar
curl -sS 'http://127.0.0.1:8101/ambiente' | jq .
```

Logs opcionais

```
./scripts/run_api_ambiente.sh 2>&1 | tee /tmp/api_ambiente.log
```

Output esperado

- Healthcheck retornando JSON com status da API
- POST retornando registro criado
- GET listando registros

PASSO 8: Configurar serviço systemd

EXECUTAR NO TERMINAL DO SERVIDOR (SSH)

O que este bloco faz

Instala o serviço da API no systemd para subir no boot e reiniciar automaticamente.

```
cd /home/deploy/var/www/sistema-gestao/cidadesverdes

# 1) Revisar e ajustar caminhos
nano systemd/api-ambiente.service

# 2) Instalar serviço
sudo cp systemd/api-ambiente.service /etc/systemd/system/api-ambiente.service

# 3) Recarregar e habilitar
sudo systemctl daemon-reload
sudo systemctl enable api-ambiente

# 4) Iniciar/reiniciar
sudo systemctl restart api-ambiente

# 5) Validar status
sudo systemctl status api-ambiente --no-pager
```

Observabilidade

```
sudo journalctl -u api-ambiente -f
sudo journalctl -u api-ambiente -n 100 --no-pager
```

Output esperado

- Loaded: loaded (...)
 - Active: active (running)
-

PASSO 9: Liberar porta 8101 no firewall

 EXECUTAR NO TERMINAL DO SERVIDOR (SSH)

O que este bloco faz

Libera acesso TCP externo para a API.

```
sudo ufw allow 8101/tcp
sudo ufw status numbered
```

Pausa para verificação

```
sudo ss -ltnp | grep 8101 || true
```

Output esperado

- Regra 8101/tcp ALLOW no UFW.
-

PASSO 10: Configurar Nginx (opcional)

• EXECUTAR NO TERMINAL DO SERVIDOR (SSH)

O que este bloco faz

Cria reverse proxy em `/api/ambiente/` apontando para `127.0.0.1:8101`.

```
sudo nano /etc/nginx/sites-available/cidadesverdes-api
```

```
server {  
    listen 80;  
    server_name SEU_DOMINIO_OU_IP;  
  
    location /api/ambiente/ {  
        proxy_pass http://127.0.0.1:8101/  
        proxy_http_version 1.1;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        proxy_set_header X-Forwarded-Proto $scheme;  
    }  
}
```

```
sudo ln -s /etc/nginx/sites-available/cidadesverdes-api /etc/nginx/sites-enabled/cid-  
adesverdes-api  
sudo nginx -t  
sudo systemctl reload nginx
```

Pausa para verificação

```
curl -i http://127.0.0.1/api/ambiente/
```

Output esperado

- `nginx -t` com `syntax is ok` e `test is successful`.

PASSO 11: Testes finais + troubleshooting

• EXECUTAR NO TERMINAL DO SERVIDOR (SSH)

O que este bloco faz

Valida operação ponta a ponta e fornece comandos de diagnóstico.

```
# 1) Serviço ativo
sudo systemctl is-active api-ambiente

# 2) Endpoint local
curl -sS http://127.0.0.1:8101/ | jq .

# 3) Endpoint externo direto
curl -sS http://SEU_IP_PUBLICO:8101/ | jq .

# 4) Endpoint externo via Nginx (se configurado)
curl -sS http://SEU_DOMINIO/api/ambiente/ | jq .

# 5) Logs recentes
sudo journalctl -u api-ambiente -n 100 --no-pager

# 6) Conferir dados no Firebird
isql-fb -user SYSDBA -password 'SENHA_AQUI' /home/deploy/var/databases/cidades-
verdes.fdb << 'SQL'
SELECT COUNT(*) FROM ambiente;
QUIT;
SQL
```

Comandos rápidos de troubleshooting

```
# Restart da API
sudo systemctl restart api-ambiente

# Restart do Firebird
sudo systemctl restart firebird3.0

# Status consolidado
sudo systemctl status api-ambiente firebird3.0 --no-pager

# Portas ativas
sudo ss -ltnp | egrep '8101|3050' || true

# Variáveis carregadas pelo systemd (debug)
sudo systemctl show api-ambiente --property=Environment
```

Ordem resumida de execução

1. Commit/push no GitHub (LOCAL)
2. `ssh + git pull` (SERVIDOR)
3. `./scripts/setup_firebird.sh`
4. Permissões em `/home/deploy/var/databases/`
5. Configurar `.env`
6. `python3 scripts/create_database.py`
7. Teste manual da API
8. Configurar `systemd`
9. Liberar UFW 8101
10. (Opcional) Nginx
11. Testes finais